

Locking Bootloader on LineageOS

https://gemini.google.com/app/8577091a0835d994

User prompt: How do i lock the bootloader on lineage os pixel?

Response: **Short answer:** If you are running standard official LineageOS builds, **do not lock your bootloader**. Running fastboot flashing lock on stock LineageOS will soft-brick or hard-brick your Pixel.

Here is why this happens, along with the *only* proper way to achieve a locked bootloader on custom software.

Why You Can't Simply Lock It

Android uses **Android Verified Boot (AVB)**. When the bootloader is locked, the hardware checks the signatures of the installed operating system against the cryptographic keys stored in the phone's secure element.

- **Stock Google Firmware:** Signed with Google's private key. The bootloader trusts it.
- **Official LineageOS:** Signed with LineageOS's keys or standard test keys.
- **The Problem:** The bootloader does not have LineageOS's key enrolled by default. If you force a lock while running LineageOS, AVB will fail, classify the OS as untrusted/tampered, and **refuse to boot** (you'll hit a bootloop or a "Your device is corrupt" red screen).

How it *Is* Possible (Custom AVB Keys)

Google Pixels are among the few devices that support **AVB 2.0 with custom key enrollment**. This allows you to set up the bootloader to trust your own cryptographic key instead of Google's.

To lock the bootloader while running LineageOS, you must:

1. **Build LineageOS from source yourself** (or use an open-source tool like avbroot).
2. **Sign the build** with your own privately generated keys.
3. **Flash your public key (pkmd.bin)** into the Pixel's bootloader memory via fastboot (fastboot flash avb_custom_key pkmd.bin).
4. **Lock the bootloader** (fastboot flashing lock).

When the phone boots, the locked bootloader compares the custom OS against the enrolled pkmd.bin key. Since they match, it boots up safely (displaying a yellow warning screen that a custom OS key is in use).

What Should You Do?

Depending on your goal, you have three options:

Option A: Leave the Bootloader Unlocked (Recommended for Standard LineageOS)

If you just want to use LineageOS, leave the bootloader unlocked. If your goal is to pass Play Integrity / SafetyNet checks for banking apps, locking the bootloader is no longer the primary requirement anyway — modern integrity checks flag custom ROM signatures regardless. Most users rely on Magisk/Play Integrity Fix modules instead.

Option B: Switch to an Out-of-the-Box Relockable ROM

If hardware-backed security and a locked bootloader are your absolute top priorities, consider switching to **GrapheneOS** or **CalyxOS**. Both are tailored specifically for Pixels and provide pre-built, automated key-enrollment scripts so you can lock the bootloader during installation without building from source.

Option C: Sign LineageOS Yourself via avbroot

If you are comfortable with command-line tools and patching images, you can use the third-party open-source utility **avbroot**. It unpacks official LineageOS zip files, patches the boot image, signs everything with your own generated key, and gives you the exact pkmd.bin file needed to lock the bootloader safely.

Sources

1. [Is it possible to relock the bootloader LOS 18.1 Pixel 4 ?? : r/LineageOS - Reddit](#)
2. [Bootloader Lock – The AXP.OS Project](#)
3. [Supported devices · Issue #299 · chenxiaolong/avbroot - GitHub](#)
4. [Lock and unlock the bootloader | Android Open Source Project](#)

User prompt: can you tell me how to sign lineageos myself?

Response: To sign LineageOS yourself so you can lock your Pixel's bootloader without bricking it, the cleanest and most reliable modern method is using **avbroot**.

Instead of compiling LineageOS from source (which takes hours and hundreds of gigabytes of disk space), avbroot unpacks official LineageOS update packages, patches the Android Verified Boot (AVB) structures, signs everything with your own generated keys, and spits out a signed package ready for your Pixel.

CRITICAL WARNING:

- **Backup everything.** Locking or unlocking the bootloader automatically **wipes all user data**.
- **Keep OEM Unlocking enabled in Developer Options.** Never disable "OEM Unlocking" in Android settings while running a custom signed OS. If something ever breaks, this setting is your safety hatch to rescue the device via fastboot.

What You Need Before Starting

- **A host computer** (Linux is recommended and fastest, but macOS/Windows work fine).
- **The avbroot tool** (downloadable as a pre-compiled binary from GitHub).
- **OpenSSL** installed on your computer.
- **android-tools** installed (adb and fastboot).
- **Official LineageOS zip file** for your specific Pixel model.

- **Official LineageOS Recovery image (recovery .img)** for your model.

The Step-by-Step Signing Process

1 Generate your private signing keys

On your computer

Create a secure directory on your computer to generate and store your custom AVB cryptographic keys.

```
mkdir ~/lineage-keys && cd ~/lineage-keys
```

```
# Generate PKCS#8 private key
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 -out avb.key
```

```
# Generate AVB public key metadata (pkmd.bin)
avbroot key extract-avb -k avb.key -o pkmd.bin
```

Important: Protect avb.key. Anyone with this key could forge software updates for your locked phone.

2 Sign the LineageOS OTA package

On your computer

Run avbroot to sign both the main installation zip and the recovery image using your generated key.

```
# Sign the recovery image (needed for flashing custom key support)
avbroot ota patch \
  --input lineage-*.zip \
  --key-avb avb.key \
  --cert-avb avb.key \
  --output lineage-signed.zip
```

```
# Extract signed recovery from the patched OTA if needed
avbroot ota extract \
  --input lineage-signed.zip \
  --directory ./extracted/
```

3 Flash custom AVB key to your Pixel

Fastboot Mode

Reboot your Pixel into Fastboot mode (adb reboot bootloader). Flash your public key (pkmd.bin) into the phone's secure hardware memory so the bootloader trusts your signature.

```
fastboot erase avb_custom_key
fastboot flash avb_custom_key pkmd.bin
```

4 Flash signed LineageOS & Recovery

Fastboot & ADB

Flash the signed recovery image, boot into recovery, and sideload your signed LineageOS zip.

```
# Flash signed recovery
fastboot flash boot extracted/boot.img
fastboot flash vendor_boot extracted/vendor_boot.img # (If applicable to your Pixel model)
```

```
# Boot into recovery on phone, select "Apply Update via ADB", then:
adb sideload lineage-signed.zip
```

5 Lock the Bootloader

Fastboot Mode

Reboot back into Fastboot mode. Now that the phone has signed software installed matching the pkmd.bin key stored in hardware, lock the bootloader.

```
fastboot flashing lock
```

On your phone's screen, use the volume keys to select "**LOCK THE BOOTLOADER**" and confirm with the power button. Your device will wipe itself and boot securely into signed LineageOS!

Managing Future Updates

Once your bootloader is locked with custom keys, **you can no longer install standard official LineageOS OTA updates directly.**

To update your phone in the future:

1. Download the new official LineageOS update .zip.
2. Run avbroot ota patch using your same avb.key.
3. Sideload the freshly signed zip via ADB recovery or use avbroot's built-in OTA app integration.

User prompt: how to install avbroot?

Response: avbroot is a **standalone executable**. You don't actually need to compile it or run a complex installer script — you simply download the binary for your operating system, make it executable, and move it into your system path.

Here is how to get it set up depending on your OS.

Quick Install (Linux / macOS)

1. Download the latest binary

Head over to the official [avbroot Releases on GitHub](#) and download the pre-compiled archive matching your system (e.g., avbroot-x86_64-unknown-linux-gnu.zip for standard Linux or aarch64-apple-darwin.zip for Apple Silicon Macs).

Alternatively, grab it via terminal:

```
# Example for Linux x86_64 (replace the version/filename with the latest release)
wget https://github.com/chenxiaolong/avbroot/releases/download/v3.29.0/avbroot-v3.29.0-x86_64-unknown-linux-gr

# Unzip the downloaded file
unzip avbroot-*-x86_64-unknown-linux-gnu.zip
```

2. Make it executable and move to PATH

To run avbroot from anywhere in your terminal without typing ./avbroot every time:

```
# Make it executable
chmod +x avbroot

# Move it to a directory in your PATH (e.g., /usr/local/bin)
sudo mv avbroot /usr/local/bin/
```

3. Verify the installation

Check that your shell recognizes the binary:

```
avbroot --version
```

Quick Install (Windows)

1. Go to the [avbroot Releases on GitHub](#).
2. Download the Windows zip file (e.g., avbroot-vX.X.X-x86_64-pc-windows-msvc.zip).
3. Extract the .zip file. You will see avbroot.exe inside.
4. **Usage Option:**
 - **Simple:** Copy avbroot.exe directly into the folder where you are storing your LineageOS files, open PowerShell / Command Prompt in that directory, and run commands using .\avbroot.exe.
 - **Global:** Add the folder containing avbroot.exe to your Windows **System Environment Variables (PATH)** so you can run avbroot from any command prompt window.

Linux Alternative: Package Managers

If you prefer using package managers, avbroot is available natively on some distributions:

- **Arch Linux (AUR):**

```
yay -S avbroot
```
- **NixOS:** Available natively in nixpkgs as avbroot.

Prerequisites to double-check

Now that avbroot is installed, make sure you also have its external dependencies ready on your system before patching your ROM:

1. **openssl** (Needed to generate private RSA signing keys).
2. **android-tools** / Platform Tools (Ensure your fastboot binary is **v35.0.0 or newer**, as older versions have known bugs when flashing extracted partition maps). Check with fastboot --version.

Sources

1. [chenxiaolong/avbroot: Tool for manipulating and re-signing Android A/B OTAs - GitHub](#)

User prompt: avbroot installed what's next?

Response: Now that avbroot is installed, you are ready for the actual execution pipeline.

 **CRITICAL REMINDER:** Flashing custom keys and locking the bootloader **will wipe your entire phone**. Make sure you have backed up any important data first.

Here is the exact end-to-end process to patch LineageOS, set your custom key, and lock the bootloader safely.

Step 1: Generate Your Custom Keys

avbroot has built-in subcommands to generate the two keys you need: an **AVB key** (for the hardware bootloader) and an **OTA key** (for sideloading updates).

Run these commands in a fresh working directory on your computer:

```
mkdir ~/lineage-signing && cd ~/lineage-signing

# 1. Generate the AVB private key and the OTA signing key
avbroot key generate-key -t rsa4096 -o avb.key
avbroot key generate-key -t rsa4096 -o ota.key

# 2. Encode the AVB public key into the format your Pixel's bootloader expects (pkmd.bin)
avbroot key encode-avb -k avb.key -o avb_pkmd.bin

# 3. Generate a self-signed certificate for the OTA key (used by recovery)
avbroot key generate-cert -k ota.key -o ota.crt
```

Security Note: Keep avb.key and ota.key safe. Do not delete them! You will need these exact same key files every time LineageOS releases an update.

Step 2: Patch the LineageOS OTA Zip

Download the official LineageOS update .zip file for your Pixel into this folder.

Run avbroot to replace LineageOS's stock signatures with your custom signatures:

```
avbroot ota patch \  
  --input lineage-*.zip \  
  --key-avb avb.key \  
  --key-ota ota.key \  
  --cert-ota ota.crt \  
  --rootless
```

(Note: If you want root access via Magisk/KernelSU instead of unrooted, you would swap --rootless for --magisk /path/to/magisk.apk).

This will output a file named **lineage-*.patched.zip**.

Step 3: Extract the Signed Boot & Recovery Images

Because you are transitioning to custom keys, you need to flash the patched boot/recovery images to your Pixel via Fastboot first. Extract them directly from your newly patched zip:

```
avbroot ota extract \  
  --input lineage-*.patched.zip \  
  --directory ./extracted/
```

Inside ./extracted/, you will see files like boot.img, vendor_boot.img (if applicable to your Pixel model), and init_boot.img (for Pixel 7 and newer).

Step 4: Flash Keys & Signed Images (Fastboot Mode)

1. Put your Pixel into **Fastboot Mode** (Power off, then hold Volume Down + Power, or run adb reboot bootloader).
2. Connect it to your computer.

Run the following commands in order:

```
# 1. Clear old keys and flash your newly generated public key to hardware  
fastboot erase avb_custom_key  
fastboot flash avb_custom_key avb_pkmd.bin  
  
# 2. Flash the patched boot/vendor_boot images from your extracted directory  
fastboot flash boot extracted/boot.img  
  
# (ONLY if these files exist in your extracted/ folder for your specific Pixel model):  
fastboot flash vendor_boot extracted/vendor_boot.img  
fastboot flash init_boot extracted/init_boot.img
```

Step 5: Sideload the Patched LineageOS Zip

1. On your phone's Fastboot menu, use the volume keys to select **Recovery Mode** and press Power.
2. In Lineage Recovery, tap **Factory Reset -> Format data / factory reset**.
3. Go back to the main recovery menu and tap **Apply update -> Apply from ADB**.
4. On your computer terminal, sideload your patched OS zip:

```
adb sideload lineage-*.patched.zip
```

Step 6: Lock the Bootloader

Once the sideload completes successfully (it may stop at 47% or 100% in terminal — as long as the phone shows "Step 2/2" complete, it succeeded):

1. Reboot the phone back into **Fastboot Mode** from the recovery menu.
2. Run the lock command:

```
fastboot flashing lock
```

3. Look at your phone's screen. Use the volume buttons to highlight **"LOCK THE BOOTLOADER"** and hit the **Power button** to confirm.

Your Pixel will wipe itself, perform its hardware check against your avb_pkmd.bin key, and boot cleanly into your signed LineageOS build with a fully locked bootloader!

Sources

1. [chenxiaolong/avbroot: Tool for manipulating and re-signing Android A/B OTAs - GitHub](#)
2. [Install LineageOS on channel](#)

User prompt: what should be the passphrase for avb key?

Response: avbroot **does not require or enforce a passphrase** on your avb.key file by default when you run avbroot key generate-key.

However, whether you *should* use a passphrase comes down to a balance between security and convenience:

1. No Passphrase (Unencrypted Key) — Recommended for Most Users

If you do not set a passphrase (or leave it blank), the key file lives on your computer as a plain .key file.

- **Pros:** Allows you to automate future updates seamlessly using scripts or avbroot's automated workflows.
- **Cons:** Anyone who gains physical or root access to your computer's filesystem can copy avb.key.
- **Why it's usually fine:** Unless you are targeting a high-threat security model where someone has physical access to your logged-in PC, keeping avb.key inside a secure user folder (or an encrypted drive like BitLocker / LUKS) is standard practice.

2. Setting a Passphrase (Encrypted Key)

If you generate an encrypted private key using `openssl` (or assign a passphrase), OpenSSL will wrap the key using AES-256 encryption.

- **Pros:** If your computer is stolen or compromised, your device key remains secure.
- **Cons:** You must **manually enter the passphrase every single time** you run `avbroot ota patch` for future monthly LineageOS updates.

The Golden Rules for Your `avb` .key

Regardless of whether you use a passphrase, follow these three rules:

1. **Back it up securely:** If you lose `avb` .key, **you can never update your phone again without wiping it**. You would have to unlock the bootloader (which triggers a full factory reset) and start over with new keys. Backup your `lineage-signing` directory to a secure encrypted flash drive or password manager vault.
2. **Never leak it:** Anyone with your `avb` .key can sign malicious updates or modified boot images that your locked phone's hardware would trust and boot.
3. **Store it locally:** Never commit `avb` .key to a public GitHub repository.

```
User prompt: avbroot ota patch \ --input lineage-*.zip \ --key-avb avb.key \ --key-ota ota.key \ --cert-ota ota.crt \ --rootless 0.001s INFO Replacing zip entry: META-INF/com/android/otacert 0.001s INFO Copying zip entry: apex_info.pb 0.001s INFO Copying zip entry: care_map.pb 0.001s INFO Patching zip entry: payload.bin 0.001s INFO Extracting from original payload: system 1.428s INFO Extracting from original payload: vbmeta_system 1.428s INFO Extracting from original payload: vbmeta 1.428s INFO Extracting from original payload: boot 1.539s INFO Candidate boot images: boot 2.274s INFO Patching system image: system 3.284s INFO Patched otacerts.zip offsets in system: [337145856..337147392] 3.284s INFO Patching vbmeta images: vbmeta_system, vbmeta 3.361s ERROR Failed to patch OTA zip Caused by: 0: Failed to patch payload: payload.bin 1: Verified boot is disabled by vbmeta's header flags: 0x3
```

Response: This error happens because official LineageOS builds ship with **Android Verified Boot (AVB) explicitly disabled** inside their `vbmeta` flags (0x3 = verification disabled & verification errors ignored).

When `avbroot` sees this, it stops by default because patching disabled flags without re-enforcing AVB won't allow you to lock the bootloader safely.

The Fix

To fix this, pass the `--clear-vbmeta-flags` flag in your command. This forces `avbroot` to strip out those disable flags and re-enable active verified boot so your hardware bootloader can actually validate the OS signatures.

Run your patch command like this instead:

```
avbroot ota patch \
--input lineage-*.zip \
--key-avb avb.key \
--key-ota ota.key \
--cert-ota ota.crt \
--clear-vbmeta-flags \
--rootless
```

What happens now?

1. `avbroot` will clear the 0x3 flag inside `vbmeta.img` and `vbmeta_system.img`.
2. It will generate a signed `lineage-*.patched.zip`.
3. You can continue with extracting the images and flashing as normal.

Sources

1. [chenxiaolong/avbroot: Tool for manipulating and re-signing Android A/B OTAs - GitHub](#)

User prompt: done what's next

Response: You now have a **patched, signed LineageOS zip** file!

Now you need to extract the signed core boot images, flash your public key to the Pixel's secure hardware, sideload the patched zip, and lock the bootloader.

Step 1: Extract the Signed Images

Extract the newly signed boot images directly from your patched zip file:

```
avbroot ota extract \
--input lineage-*.patched.zip \
--directory ./extracted/
```

Look inside the `./extracted/` folder. Depending on your Pixel model, you will see `boot.img`, along with `vendor_boot.img` or `init_boot.img`.

Step 2: Flash Key & Images (Fastboot Mode)

1. Boot your Pixel into **Fastboot Mode** (Power off, hold `Volume Down` + `Power`, or run `adb reboot bootloader` from Android).
2. Connect it to your PC.

Run these fastboot commands to flash your public key into hardware and install the signed boot partitions:

```
# 1. Clear any old key and flash your public key to hardware
fastboot erase avb_custom_key
fastboot flash avb_custom_key avb_pkmd.bin

# 2. Flash the signed boot image
fastboot flash boot extracted/boot.img
```



```
# 3. Flash extra partitions IF they exist in your extracted/ folder:
fastboot flash vendor_boot extracted/vendor_boot.img
fastboot flash init_boot extracted/init_boot.img
```

Step 3: Sideload the Patched Zip

1. On your Pixel screen, use the Volume buttons to select **Recovery Mode** and press Power.
2. In Lineage Recovery, tap **Factory Reset -> Format data / factory reset** to clear old data.
3. Return to the main Recovery menu and tap **Apply update -> Apply from ADB**.
4. Sideload your patched zip from terminal:

```
adb sideload lineage-*.patched.zip
```

(It is normal for adb terminal progress to pause around 47% — as long as the phone screen says “Step 2/2” or “Done”, it succeeded!)

Step 4: Lock the Bootloader

1. From the Recovery menu, select **Advanced -> Reboot to bootloader**.
2. Run the lock command:

```
fastboot flashing lock
```

3. Look at your Pixel's screen. Use the volume buttons to highlight **"LOCK THE BOOTLOADER"** and hit the **Power button**.

Your phone will wipe itself, check its software signatures against your hardware-enrolled avb_pkmd.bin key, and boot cleanly into your signed LineageOS!

Sources

1. [chenxiaolong/avbroot: Tool for manipulating and re-signing Android A/B OTAs - GitHub](#)
2. [Install LineageOS on Google Pixel 5 - redfin](#)
3. [Install LineageOS on Google Pixel XL - marlin](#)

User prompt: preyas@preyas-960QFG:~/lineage-signing\$ avbroot ota extract \ --input lineage-*.patched.zip \ --directory ./extracted/ 0.000s
ERROR Failed to open for reading: "lineage-*.patched.zip" Caused by: No such file or directory (os error 2) preyas@preyas-960QFG:~/lineage-signing\$ ls avb.key avb_pkmd.bin boot.img dtbo.img lineage-23.2-20260717-nightly-coral-signed lineage-23.2-20260717-nightly-coral-signed.zip lineage-23.2-20260717-nightly-coral-signed.zip.patched ota.crt ota.key super_empty.img vbmeta.img

Response: Your filename ends with .zip.patched instead of .patched.zip. Because of that, the wildcards (lineage-*.patched.zip) didn't find it.

Here is the exact command using your file's actual name:

```
avbroot ota extract \
--input lineage-23.2-20260717-nightly-coral-signed.zip.patched \
--directory ./extracted/
```

Once that runs, check inside ./extracted/ to see what images were generated.

Since you are on **Pixel 4 XL (coral)**, the critical image files you will need for Step 2 are:

- boot.img
- dtbo.img
- vbmeta.img

Once extracted, you can jump straight into **Step 2 (Fastboot Flashing)**!

User prompt: ls avb.key avb_pkmd.bin boot.img dtbo.img lineage-23.2-20260717-nightly-coral-signed lineage-23.2-20260717-nightly-coral-signed.zip lineage-23.2-20260717-nightly-coral-signed.zip.patched ota.crt ota.key super_empty.img vbmeta.img preyas@preyas-960QFG:~/lineage-signing\$ avbroot ota extract \ --input lineage-23.2-20260717-nightly-coral-signed.zip.patched \ --directory ./extracted/ 0.000s
INFO Extracting from the payload: boot, system, vbmeta, vbmeta_system 1.532s INFO Successfully extracted images from payload preyas@preyas-960QFG:~/lineage-signing\$ # 1. Clear any old key and flash your public key to hardware fastboot erase avb_custom_key fastboot flash avb_custom_key avb_pkmd.bin # 2. Flash the signed boot image fastboot flash boot extracted/boot.img # 3. Flash extra partitions IF they exist in your extracted/ folder: fastboot flash vendor_boot extracted/vendor_boot.img fastboot flash init_boot extracted/init_boot.img Erasing 'avb_custom_key' OKAY [0.414s] Finished. Total time: 0.500s Warning: skip copying avb_custom_key image avb footer (avb_custom_key partition size: 0, avb_custom_key image size: 1032). Sending 'avb_custom_key' (1 KB) OKAY [0.140s] Writing 'avb_custom_key' OKAY [0.228s] Finished. Total time: 0.778s Sending 'boot_a' (65536 KB) OKAY [0.350s] Writing 'boot_a' OKAY [0.437s] Finished. Total time: 1.195s fastboot: error: cannot load 'extracted/vendor_boot.img': No such file or directory fastboot: error: cannot load 'extracted/init_boot.img': No such file or directory preyas@preyas-960QFG:~/lineage-signing\$ LS LS: command not found preyas@preyas-960QFG:~/lineage-signing\$ LS LS: command not found preyas@preyas-960QFG:~/lineage-signing\$ ls avb.key avb_pkmd.bin boot.img dtbo.img extracted lineage-23.2-20260717-nightly-coral-signed lineage-23.2-20260717-nightly-coral-signed.zip lineage-23.2-20260717-nightly-coral-signed.zip.patched ota.crt ota.key super_empty.img vbmeta.img preyas@preyas-960QFG:~/lineage-signing\$

Response: Everything actually went **100% smoothly**!

The "errors" for vendor_boot.img and init_boot.img are completely normal and expected — those partitions don't exist on the Pixel 4 XL (coral). They were introduced on newer Pixels (Pixel 6+). Your avb_custom_key and boot.img flashed successfully.

Here is your exact next step: **Boot to Lineage Recovery and Sideload**.

Step 1: Boot into Recovery Mode

1. On your Pixel 4 XL, use the **Volume buttons** to scroll through the bootloader menu until you see **"Recovery Mode"**.
2. Press the **Power button** to select it.

Step 2: Wipe Data & Start Sideload

1. Once Lineage Recovery loads:

- Tap **Factory Reset -> Format data / factory reset**. (This clears old keys/data to prevent boot loops).
2. Go back to the main Recovery menu:
- Tap **Apply update -> Apply from ADB**.

Step 3: Sideload Your Patched Zip

On your terminal, run:

```
adb sideload lineage-23.2-20260717-nightly-coral-signed.zip.patched
```

(Note: Terminal progress usually pauses around 47% or 100%. As long as your phone's screen displays "Step 2/2" or "Done", the installation succeeded).

Step 4: Lock the Bootloader

1. On your phone, tap **Advanced -> Reboot to bootloader**.
2. Back in Fastboot mode, run:

```
fastboot flashing lock
```

3. Look at your Pixel screen, use the Volume keys to highlight "**LOCK THE BOOTLOADER**", and press Power to confirm.

Your Pixel 4 XL will wipe, perform its AVB key check against `avb_pkmd.bin`, and boot securely into signed LineageOS 23.2!

User prompt: *what about android apps? Do i need to reinstall them as well?*

Response: **Yes, you will need to reinstall and reconfigure your apps.**

When you ran `fastboot flashing lock` in the final step, the bootloader automatically triggered a **full factory wipe** of your device. This is a hardware-level Android security feature designed to prevent someone from locking/unlocking a stolen phone to gain unauthorized access to existing user data.

What Happens Now?

- **OS Level:** LineageOS 23.2 is signed and installed.
- **Bootloader Level:** Secured with your custom hardware key (`avb_pkmd.bin`).
- **Storage Level:** Completely wiped. The phone will boot into the initial LineageOS setup screen.

Restoring Your Apps

1. **Complete Initial Setup:** Go through the LineageOS setup wizard.
2. **Google Apps (If using GApps):** If you flashed a GApps package (or microG) alongside LineageOS, log into your Google Account. Your cloud-backed-up apps will re-download automatically.
3. **App Stores:** If running pure, de-Googled LineageOS, open **Aurora Store** or **F-Droid** to install your apps.

💡 Important Pro-Tip for Future Updates

Now that your bootloader is locked with your custom key:

- **DO NOT wipe data again for normal updates.**
- When LineageOS releases a monthly update, you simply patch the new `.zip` with `avbroot` using your **same `avb.key` and `ota.key`** and sideload it.
- Because the keys match, your phone will update in place **without losing your apps or data!**

Sources

1. [chenxiaolong/avbroot: Tool for manipulating and re-signing Android A/B OTAs - GitHub](#)
2. [Step-by-Step Instructions to Relock the Bootloader on Google Pixel \(Android 14\) - Cellmigo](#)
3. [Back up user data with Auto Backup | Identity - Android Developers](#)

User prompt: *the google play store and stuff*

Response: Because LineageOS is a lightweight, open-source build, standard Google apps (Google Play Store, Play Services, Maps, etc.) are **not included by default**.

If you want Google Play Store on your Pixel 4 XL, you have two ways to handle it:

Option A: MicroG via Magisk (Recommended with avbroot)

Because your bootloader is locked with custom keys, directly sideloading standard GApps (like *MindTheGapps*) post-install breaks system integrity verification and can cause boot loops.

The standard approach when using `avbroot` is to flash **MicroG** via Magisk:

1. Re-patch your LineageOS OTA zip with `avbroot` using the `--magisk` flag to embed root access.
2. Sideload the patched zip.
3. Install the **Play Integrity Fix** and **MicroG / GApps Systemless** modules inside Magisk.

This gives you full Google Play Store functionality while keeping your bootloader securely locked.

Option B: Aurora Store (No Root Required)

If you don't want to root your phone or deal with patching Magisk:

1. Open the pre-installed **Browser** on LineageOS.
2. Go to `f-droid.org` or `auroraoss.com` and download **Aurora Store**.
3. Aurora Store is an open-source frontend for the Google Play Store. It allows you to log in (or use an anonymous account) to download and update **every single app from the official Google Play Store** without needing full Play Services running in the background.

Note on Banking/Pay Apps: If you rely on **Google Wallet/Pay** or strict banking apps that require *Play Integrity STRONG/HARDWARE evaluation*, even with a locked bootloader, Google flags custom ROM signatures. Modules like *Play Integrity Fix* are required to bypass these checks.

Sources

1. [Lock bootloader : r/LineageOS - Reddit](#)
2. [chenxiaolong/avbroot: Tool for manipulating and re-signing Android A/B OTAs - GitHub](#)

User prompt: *im planning to use mind theGapps, which version will it be 14 or 16?*

Response: Based on the file name you are flashing (`lineage-23.2-20260717-nightly-coral-signed.zip.patched`), you will need **MindTheGapps 16**.

Here is how the versioning translates:

- **LineageOS 21** = Android 14
- **LineageOS 22** = Android 15
- **LineageOS 23 & 23.2** = **Android 16**

Always match the major Android version of MindTheGapps to the underlying OS version. Since LineageOS 23.2 is built on Android 16, grab the **MindTheGapps 16 (arm64)** variant for your Pixel 4 XL.

 **Important Reminder for avbroot:** If you plan to install MindTheGapps, remember that sideloading an unpatched/unsigned GApps zip on top of an avbroot-signed ROM will trigger a bootloop or verification failure under a locked bootloader.

You will either need to patch the MindTheGapps zip alongside your OS update using avbroot or use the MicroG/Aurora method mentioned earlier.

Sources

1. [LineageOS - Wikipedia](#)
2. [LineageOS 23.2 | Android 16 QPR2 | OnePlus 6 & 6T - YouTube](#)

User prompt: */home/preyas/Downloads/MindTheGapps-16.0.0-arm64-20260409_073023.zip how will we install this?*

Response: Because your bootloader is locked with custom keys, **you cannot simply adb sideLoad an unpatched MindTheGapps zip**. Standard MindTheGapps is not signed with your `avb.key` or `ota.key`, so Lineage Recovery and Android Verified Boot (AVB) will reject it and trigger a boot failure.

To install MindTheGapps on a locked avbroot system, you have two options depending on whether you want root access or not:

Option 1: Systemless via Magisk / KernelSU (Easiest)

GApps modifies system partitions. With a locked bootloader, modifying system files directly breaks verification. By using Magisk, GApps is mounted "systemlessly" at boot while keeping your custom AVB partition hashes clean.

1. **Re-patch LineageOS with Magisk:** Run avbroot to embed Magisk into your LineageOS zip:

```
avbroot ota patch \  
  --input lineage-23.2-20260717-nightly-coral-signed.zip \  
  --key-avb avb.key \  
  --key-ota ota.key \  
  --cert-ota ota.crt \  
  --magisk /path/to/magisk.apk
```

2. **Sideload the newly patched zip:** Reboot to Recovery (`adb reboot recovery`) and sideload `lineage-*.patched.zip`.
3. **Boot into LineageOS & Open Magisk:** Complete setup, open the Magisk app, and finish the initial environment setup.
4. **Install GApps Module:** Use a tool like **MindTheGapps-Magisk** or convert the MindTheGapps zip to a Magisk module (using the *GApps-to-Magisk* converter) and flash it directly inside the Magisk app under **Modules -> Install from storage**.
5. **Reboot:** You will have full Google Play Services running systemlessly!

Option 2: Pre-mount GApps directly into the System Partition (Advanced)

If you want a **100% rootless** setup (no Magisk installed) with Google Play Services integrated into the ROM before locking, avbroot allows you to inject system modifications or modules directly into the payload using custom build/repack scripts.

This requires extracting `system.img`, merging the MindTheGapps files into `/system`, and using avbroot to re-sign the modified `system.img` with your `avb.key` before packing it back into the OTA zip.

Which path do you prefer?

- **Path A (Magisk/Systemless):** Takes 5 minutes, easier to maintain, recommended for avbroot users.
- **Path B (Rootless Injection):** Keeps the OS 100% unrooted, but requires extra command-line unpacking/re-signing steps.

Sources

1. [chenxiaolong/avbroot: Tool for manipulating and re-signing Android A/B OTAs - GitHub](#)

User prompt: *can you tell me the prcedure to install lineage os onto a device, this fresh signed version, so I can create a document*

Response: I have created the document with the complete procedure for installing your custom signed LineageOS build and relocking the bootloader.

[LineageOS Custom Signed Installation & Bootloader Relocking Guide](#)

[file-tag: drive-uploaded-file-1VBDrMX3LoYidkZTPsN98wigl8rP4T-iNsjwQhuDxiM]

Overview of Included Steps

- 1. Key Enrollment & Partition Flashing (Fastboot Mode):**
 - Erase old key: `fastboot erase avb_custom_key`
 - Flash hardware key: `fastboot flash avb_custom_key avb_pkmd.bin`
 - Flash signed boot image: `fastboot flash boot extracted/boot.img` (and `vendor_boot/init_boot` if applicable)
- 2. Factory Reset & Recovery Sideload:**
 - Boot into Lineage Recovery via volume keys.
 - Perform **Factory Reset -> Format data / factory reset**.
 - Navigate to **Apply update -> Apply from ADB**.
 - Sideload package: `adb sideload lineage-*.patched.zip`
- 3. Bootloader Relocking:**
 - Reboot back into Fastboot mode (**Advanced -> Reboot to bootloader**).
 - Run locking command: `fastboot flashing lock`
 - Confirm **LOCK THE BOOTLOADER** using the physical volume/power buttons.

User prompt: also tell me how to add play store apps

Response: To get Google Play Store apps onto your locked LineageOS build, you have three distinct methods. Because your bootloader is locked with custom keys, method choice depends on whether you want native Google Play Services, open-source alternatives, or root-based integration.

Method 1: Aurora Store (Easiest & No Root/GApps Required)

Aurora Store is an open-source, full-featured client for the official Google Play Store. It downloads apps directly from Google's servers without needing Google Play Services running on your phone.

1. Open the pre-installed **Browser** on LineageOS.
2. Go to [auroraoss.com](#) (or download it via [F-Droid](#)).
3. Download and install the **Aurora Store APK**.
4. Open Aurora Store, complete the initial setup, and choose either:
 - **Anonymous Session:** Access, download, and update free Play Store apps without logging into a Google Account.
 - **Google Account Login:** Log in with your Google account to access your previously purchased or account-linked apps.

Method 2: MicroG via Magisk (Native Play Store with Systemless Root)

If you need Google Play Services background features (like push notifications for specific apps, Google accounts, or paid Play Store apps) under a locked bootloader, the standard avbroot method is using **MicroG** via Magisk.

- 1. Patch LineageOS with Magisk:** Re-patch your base LineageOS zip using avbroot with the --magisk flag:

```
avbroot ota patch \  
--input lineage-23.2-20260717-nightly-coral-signed.zip \  
--key-avb avb.key \  
--key-ota ota.key \  
--cert-ota ota.crt \  
--clear-vbmeta-flags \  
--magisk /path/to/magisk.apk
```
- 2. Sideload the newly patched zip:** Boot into Lineage Recovery and run `adb sideload lineage-*.patched.zip`.
- 3. Set up MicroG:** Open the Magisk app in LineageOS, grant required permissions, and install the **MicroG Installer** or **GApps Systemless** module directly inside Magisk under **Modules -> Install from storage**.
4. Reboot your device to access Google Play Services functionality systemlessly.

Method 3: Direct APK Sideload

For standalone apps or third-party stores, you can download .apk installation files directly:

1. Go to **Settings -> Apps -> Special app access -> Install unknown apps**.
2. Select your browser (or file manager) and toggle **Allow from this source**.
3. Download .apk files directly from reputable sources like **APKMirror** or **F-Droid** and tap to install.

User prompt: tell me how to use 2nd method

Response: Here is the step-by-step procedure for using **Method 2 (MicroG with Magisk)** to run Google Play Services on a locked avbroot system.

This method roots the phone *systemlessly*, allowing MicroG to handle Google Play Services without breaking Android Verified Boot (AVB).

Step 1: Patch LineageOS with Magisk using avbroot

You need to embed Magisk directly into your LineageOS package during the signing phase.

1. Download the latest **Magisk APK** to your workspace folder on your computer.
2. Run avbroot with the --magisk flag added to your patching command:

```
avbroot ota patch \  
--input lineage-23.2-20260717-nightly-coral-signed.zip \  
--key-avb avb.key \  
--key-ota ota.key \  
--cert-ota ota.crt \  
--magisk Magisk-v*.apk
```

3. Extract the new patched boot image:

```
avbroot ota extract \  
--input lineage-*.patched.zip \  
--directory ./extracted/
```

Step 2: Flash and Sideload

1. Reboot your phone to **Fastboot Mode** (adb reboot bootloader).
2. Flash the updated boot image:

```
fastboot flash boot extracted/boot.img
```

3. Boot into **Lineage Recovery** on your device, navigate to **Apply update -> Apply from ADB**, and sideload the newly patched OS zip:

```
adb sideload lineage-*.patched.zip
```

4. Reboot into LineageOS.

Step 3: Configure Magisk & Signature Spoofing

1. Open the **Magisk** app on your phone. It will ask to perform additional setup/environment fixes. Tap **OK** and let the phone reboot.
2. After rebooting, open **Magisk** again, tap the **Gear icon** (Settings) in the top right, and enable **Zygisk**.

Step 4: Install MicroG & Play Store

1. Download the `microg_installer_revived` Magisk module zip (from GitHub).
2. Open the **Magisk** app, go to the **Modules** tab at the bottom, select **Install from storage**, and select the `microg_installer_revived.zip` file.
3. Reboot your device.
4. Open the newly added **microG Settings** app from your app drawer.
5. Tap **Self-Check** and verify that all checkboxes (including **Signature Spoofing**) are checked.
6. Turn on **Google Account**, **Google Device Registration**, and **Cloud Messaging**.
7. Open the **Google Play Store** or **Aurora Store** app, sign in with your Google account, and install any Play Store apps!

User prompt: can you give me the full list of commands that i should run to flash a pixel 2xl bonito

Response:

Quick note on codenames: **bonito** is the codename for the **Google Pixel 3a XL**. (If you are working with an actual **Pixel 2 XL**, its codename is **taimen**).

The command workflow below is tailored for **bonito** (Pixel 3a XL) using the custom-signed avbroot pipeline.

Complete Terminal Command Sequence

Run all commands on your host workstation terminal inside your lineage-signing directory.

Phase 1: Key Generation & OTA Patching

```
# 1. Create and enter working directory  
mkdir ~/lineage-signing && cd ~/lineage-signing  
  
# 2. Download latest official LineageOS zip and Magisk (if using root/MicroG)  
# (Place lineage-*.zip and Magisk.apk into this folder)  
  
# 3. Generate private keys and public metadata  
avbroot key generate-key -t rsa4096 -o avb.key  
avbroot key generate-key -t rsa4096 -o ota.key  
avbroot key encode-avb -k avb.key -o avb_pkmd.bin  
avbroot key generate-cert -k ota.key -o ota.crt  
  
# 4. Patch the LineageOS zip file (Rootless or with Magisk)  
# For Rootless:  
avbroot ota patch \  
--input lineage-*.zip \  
--key-avb avb.key \  
--key-ota ota.key \  
--cert-ota ota.crt \  
--rootless  
  
# OR For Magisk Root (needed if running MicroG/Play Store module):  
avbroot ota patch \  
--input lineage-*.zip \  
--key-avb avb.key \  
--key-ota ota.key \  
--cert-ota ota.crt \
```

```
--magisk Magisk-*.apk

# 5. Extract signed boot image from the patched output
avbroot ota extract \
  --input lineage-*.patched.zip \
  --directory ./extracted/
```

Phase 2: Hardware Key Enrollment & Partition Flashing

Connect your device via USB:

```
# 1. Reboot device to Fastboot mode
adb reboot bootloader

# 2. Clear stock key and flash custom hardware key (pkmd.bin)
fastboot erase avb_custom_key
fastboot flash avb_custom_key avb_pkmd.bin

# 3. Flash patched boot image
# Note: Pixel 3a XL (bonito) uses 'boot.img'. It does not use 'vendor_boot' or 'init_boot'.
fastboot flash boot extracted/boot.img
```

Phase 3: Recovery Sideload & Bootloader Relocking

```
# 1. Boot into Recovery mode on device
# (On phone screen: Use Volume keys to select "Recovery Mode" and press Power button)

# 2. Format Data via Phone Interface:
# Tap "Factory Reset" -> "Format data / factory reset"
# Then tap "Apply update" -> "Apply from ADB"

# 3. Sideload patched LineageOS package
adb sideload lineage-*.patched.zip

# 4. Once complete, reboot back to Bootloader via Phone Interface:
# Tap "Advanced" -> "Reboot to bootloader"

# 5. Relock the bootloader hardware
fastboot flashing lock
```

On your phone screen, use the **Volume keys** to highlight **"LOCK THE BOOTLOADER"** and hit the **Power button** to confirm. Your device will perform a final wipe, verify the custom `avb_pkmd.bin` key, and boot cleanly into signed LineageOS!

User prompt: `avbroot ota patch \ --input lineage-*.zip \ --key-avb avb.key \ --key-ota ota.key \ --cert-ota ota.crt \ --magisk Magisk-*.apk` 0.003s
ERROR Failed to create Magisk boot image patcher Caused by: 0: Failed to open file: "Magisk-*.apk" 1: No such file or directory (os error 2)

Response: This error happens because your terminal shell cannot find a literal file named `Magisk-*.apk`.

While shells like Bash or Zsh understand wildcard asterisks (*), avbroot itself requires the **exact, literal filename** of your Magisk APK. If the file is missing from your working directory or the wildcard doesn't resolve, avbroot throws `os error 2` (No such file or directory).

The Fix

1. Check your folder

List the files in your directory to see the exact name of your Magisk APK:

```
ls -l Magisk*
```

2. Run the command with the exact filename

Replace `Magisk-*.apk` with the actual filename (e.g., `Magisk-v28.0.apk`) or pass the relative path:

```
avbroot ota patch \
  --input lineage-*.zip \
  --key-avb avb.key \
  --key-ota ota.key \
  --cert-ota ota.crt \
  --clear-vbmeta-flags \
  --magisk Magisk-v28.0.apk
```

(Remember to keep `--clear-vbmeta-flags` in the command so it handles LineageOS's disabled AVB header flags as well!)

User prompt: `magisk` doesnt exist...

Response: You just need to download the official Magisk .apk file into your lineage-signing working directory first!

Here is how to get it via terminal:

```
# 1. Make sure you are inside your signing directory
cd ~/lineage-signing

# 2. Download the latest Magisk release from official GitHub
wget https://github.com/topjohnwu/Magisk/releases/download/v30.7/app-release.apk -O Magisk-v30.7.apk
```

(If `wget` isn't installed, you can use `curl -L [https://github.com/topjohnwu/Magisk/releases/download/v30.7/app-release.apk] (https://github.com/topjohnwu/Magisk/releases/download/v30.7/app-release.apk) -o Magisk-v30.7.apk` or manually download the APK from the official [topjohnwu/Magisk GitHub releases](#) page).

Run the avbroot command again

Now that `Magisk-v30.7.apk` exists in your folder, run avbroot:

```
avbroot ota patch \
--input lineage-*.zip \
--key-avb avb.key \
--key-ota ota.key \
--cert-ota ota.crt \
--magisk Magisk-v30.7.apk
```

It will now locate the APK, patch the boot image structure, sign the zip file, and output your signed, root-ready lineage-*.patched.zip!

User prompt: s -l Magisk* -rw-rw-r-- 1 preyas preyas 0 Jul 21 21:51 Magisk-v30.7.apk preyas@preyas-960QFG:~/Downloads/Pixel3a-xl-bonito\$ avbroot ota patch \ --input lineage-*.zip \ --key-avb avb.key \ --key-ota ota.key \ --cert-ota ota.crt \ --magisk Magisk-v30.7.apk 0.000s ERROR Failed to create Magisk boot image patcher Caused by: 0: Failed to open zip file: "Magisk-v30.7.apk" 1: Missing end of central directory

Response: The error Missing end of central directory means the Magisk-v30.7.apk file on your system is empty (0 bytes), as shown by the 0 in your ls -l output:

```
-rw-rw-r-- 1 preyas preyas 0 Jul 21 21:51 Magisk-v30.7.apk
```

This happens when a download fails or gets interrupted, leaving an empty placeholder file that avbroot cannot open as a valid zip archive.

Step 1: Remove the corrupted 0-byte file

Remove the empty file from your folder:

```
rm Magisk-v30.7.apk
```

Step 2: Download the official Magisk APK

Download the official release directly using curl (using the -L flag so it follows GitHub redirect links):

```
curl -L https://github.com/topjohnwu/Magisk/releases/download/v30.7/app-release.apk -o Magisk-v30.7.apk
```

Verify that the file size is non-zero (it should be around 11–12 MB):

```
ls -lh Magisk-v30.7.apk
```

Step 3: Run avbroot again

Now that the file is complete, rerun your patching command:

```
avbroot ota patch \
--input lineage-*.zip \
--key-avb avb.key \
--key-ota ota.key \
--cert-ota ota.crt \
--magisk Magisk-v30.7.apk
```

User prompt: s -lh Magisk-v30.7.apk -rw-rw-r-- 1 preyas preyas 9 Jul 21 21:53 Magisk-v30.7.apk preyas@preyas-960QFG:~/Downloads/Pixel3a-xl-bonito\$ ls -lh Magisk-v30.7.apk -rw-rw-r-- 1 preyas preyas 12M Jul 21 21:57 Magisk-v30.7.apk preyas@preyas-960QFG:~/Downloads/Pixel3a-xl-bonito\$ avbroot ota patch --input lineage-*.zip --key-avb avb.key --key-ota ota.key --cert-ota ota.crt --magisk Magisk-v30.7.apk 0.002s ERROR Failed to create Magisk boot image patcher Caused by: Validation error: Magisk version 30700 (25211..) requires a preinit device to be specified preyas@preyas-960QFG:~/Downloads/Pixel3a-xl-bonito\$

Response: avbroot and newer Magisk versions (v27+) require you to explicitly specify the **preinit partition** where Magisk stores its setup data.

On Google Pixel devices (including Pixel 3a XL bonito), the preinit partition is almost always **metadata**.

The Fixed Command

Pass the --magisk-preinit-device metadata parameter:

```
avbroot ota patch \
--input lineage-*.zip \
--key-avb avb.key \
--key-ota ota.key \
--cert-ota ota.crt \
--magisk Magisk-v30.7.apk \
--magisk-preinit-device metadata
```

This will bypass the validation check, patch the boot image structure, sign the zip, and generate your lineage-*.patched.zip!

User prompt: avbroot ota patch \ --input lineage-*.zip \ --key-avb avb.key \ --key-ota ota.key \ --cert-ota ota.crt \ --magisk Magisk-v30.7.apk \ --magisk-preinit-device metadata

Response: That looks like the exact command structure needed! Go ahead and run it in your terminal.

What to Expect Next:

- Processing Output:** avbroot will decompress the OTA payload, patch the boot image with Magisk, re-sign the internal images with your avb.key and ota.key, and generate:
 - lineage-21.0-xxxxxxx-nightly-bonito-signed.zip.patched (or similar depending on your exact version name).
- Extracting the Patched Boot Image:** Once patching completes successfully, extract the updated boot image:

```
avbroot ota extract \
--input lineage-*.patched.zip \
--directory ./extracted/
```

- Flash to bonito:** After extracting, boot your Pixel 3a XL into Fastboot mode and flash the components:

```
adb reboot bootloader
fastboot erase avb_custom_key
```

```
fastboot flash avb_custom_key avb_pkmd.bin
fastboot flash boot extracted/boot.img
```

From there, you can proceed with booting into recovery and running `adb sideload lineage-*.patched.zip!`

```
User prompt: avbroot ota patch \ --input lineage-*.zip \ --key-avb avb.key \ --key-ota ota.key \ --cert-ota ota.crt \ --magisk Magisk-v30.7.apk \ --
magisk-preinit-device metadata 0.003s INFO Replacing zip entry: META-INF/com/android/otacert 0.003s INFO Copying zip entry: apex_info.pb
0.003s INFO Copying zip entry: care_map.pb 0.003s INFO Patching zip entry: payload.bin 0.004s INFO Extracting from original payload: system
1.642s INFO Extracting from original payload: vbmeta 1.643s INFO Extracting from original payload: boot 1.763s INFO Candidate boot images:
boot 3.177s INFO Patching system image: system 4.320s INFO Patched otacerts.zip offsets in system: [328015872..328017408] 4.320s INFO
Patchig vbmeta images: vbmeta 4.380s ERROR Failed to patch OTA zip Caused by: 0: Failed to patch payload: payload.bin 1: Verified boot is
disabled by vbmeta's header flags: 0x3 preyas@preyas-960QFG:~/Downloads/Pixel3a-xl-bonito$ avbroot ota extract \ --input lineage-*.patched.zip \
--directory ./extracted/ 0.001s ERROR Failed to open for reading: "lineage-*.patched.zip" Caused by: No such file or directory (os error 2)
preyas@preyas-960QFG:~/Downloads/Pixel3a-xl-bonito$
```

Response: The patching failed because LineageOS builds often come with Android Verified Boot (AVB) flags disabled in vbmeta by default (0x3).

avbroot prevents you from signing an OTA with those flags set because a locked bootloader will refuse to boot if AVB verification is disabled. To fix this and enable AVB, you just need to append **--clear-vbmeta-flags** to your patch command.

Step 1: Run avbroot with --clear-vbmeta-flags

```
avbroot ota patch \
--input lineage-*.zip \
--key-avb avb.key \
--key-ota ota.key \
--cert-ota ota.crt \
--magisk Magisk-v30.7.apk \
--magisk-preinit-device metadata \
--clear-vbmeta-flags
```

Step 2: Extract the freshly patched boot image

Now that the patched zip will actually be generated, extract the boot image from it:

```
avbroot ota extract \
--input lineage-*.patched.zip \
--directory ./extracted/
```

Step 3: Flash to bonito

Reboot your Pixel 3a XL into Fastboot mode and flash the updated files:

```
adb reboot bootloader
fastboot erase avb_custom_key
fastboot flash avb_custom_key avb_pkmd.bin
fastboot flash boot extracted/boot.img
```

From there, boot into Recovery mode and run:

```
adb sideload lineage-*.patched.zip
```

```
User prompt: what about google apps?
```

Response: Because your bootloader will be locked with custom AVB keys, **you cannot simply flash or sideload standard MindTheGapps.zip**.

Standard MindTheGapps is not signed with your avb.key or ota.key. If you attempt to sideload it via recovery under a locked bootloader, Android Verified Boot (AVB) will reject the signature and trigger a bootloop or verification error.

To run Google Apps (GApps) on a locked avbroot system, you have **two main approaches**:

Method 1: Systemless GApps via Magisk (Recommended)

Since avbroot natively supports patching Magisk into the system, running GApps systemlessly prevents system partition modification while keeping your custom AVB hashes intact.

Step 1: Patch LineageOS with Magisk & AVB Fixes

Run your avbroot patch command including Magisk, the preinit device, and clearing the vbmeta flags:

```
avbroot ota patch \
--input lineage-*.zip \
--key-avb avb.key \
--key-ota ota.key \
--cert-ota ota.crt \
--magisk Magisk-v30.7.apk \
--magisk-preinit-device metadata \
--clear-vbmeta-flags
```

Step 2: Extract, Flash, and Sideload

- 1. Extract the patched boot image:

```
avbroot ota extract --input lineage-*.patched.zip --directory ./extracted/
```

- 2. Flash and lock:

```
adb reboot bootloader
fastboot erase avb_custom_key
fastboot flash avb_custom_key avb_pkmd.bin
fastboot flash boot extracted/boot.img
fastboot flashing lock
```

- 3. Sideload the ROM: Boot into Lineage Recovery and run:


```
adb sideload lineage-*.patched.zip
```

Step 3: Install GApps Module in Magisk

1. Boot into LineageOS and open the **Magisk** app to complete environment setup.
2. Enable **Zygisk** in Magisk Settings.
3. Download a systemless GApps module (such as **MindTheGapps-Magisk** or **LiteGApps**) or **MicroG Revived**.
4. Install it under **Magisk -> Modules -> Install from storage** and reboot.

Method 2: Aurora Store & MicroG (No Root / Pure OS)

If you prefer to avoid rooting or messing with Magisk modules:

1. Follow the standard avbroot process **without** adding the `--magisk` flag.
2. Boot into LineageOS, open the browser, and download **Aurora Store** (from `auroraoss.com` or F-Droid).
3. Aurora Store acts as an open-source client for the Google Play Store, allowing you to log into your Google Account (or browse anonymously) and install all Play Store apps directly without needing standard GApps flashed onto your system.

User prompt: *i believe we used the magisk methodology so I just transfer this apk onto the device right?*

Response: Yes, exactly! Because you patched the ROM using the `--magisk` flag with avbroot, **Magisk is already built into your boot partition**.

However, the boot image only contains the background daemon, not the user interface. Installing the APK acts as the frontend management app to control it.

Here is what you need to do:

1. Install the Magisk App

1. Transfer the `Magisk-v30.7.apk` file to your phone (via USB file transfer or `adb install Magisk-v30.7.apk`).
2. Open the file on your device to install the app.

2. Complete the Initial Setup

1. Open the **Magisk** app from your launcher.
2. It will prompt you with a message saying **"Requires Additional Setup"**.
3. Tap **OK**. The app will automatically finish configuring the root environment and reboot your phone.

3. Install your GApps Module

Once the phone reboots, open Magisk again:

1. Go to **Settings** (gear icon) and toggle on **Zygisk**.
2. Go to the **Modules** tab at the bottom.
3. Tap **Install from storage** and select your systemless GApps module zip file (e.g., *MindTheGapps-Magisk* or *MicroG Revived*) to flash it.
4. Reboot one final time, and your Google Play services will be up and running!

User prompt: *should I restart into recovery?*

Response: **Yes, exactly**. After successfully running the `avbroot ota patch` command with `--clear-vbmeta-flags`, your next step is to get into Lineage Recovery on your Pixel 3a XL (bonito) to perform the factory reset and sideload the new patched zip.

Here is the exact terminal sequence from where you are right now:

1. Extract the patched boot image

First, extract `boot.img` from the newly generated patched OTA file on your computer:

```
avbroot ota extract \  
  --input lineage-*.patched.zip \  
  --directory ./extracted/
```

2. Flash hardware key & boot image (Fastboot Mode)

Reboot your phone into Fastboot mode and flash the necessary components:

```
# Boot to bootloader  
adb reboot bootloader  
  
# Clear existing keys and enroll your custom hardware key  
fastboot erase avb_custom_key  
fastboot flash avb_custom_key avb_pkmd.bin  
  
# Flash the patched boot partition  
fastboot flash boot extracted/boot.img
```

3. Restart into Recovery Mode

Use the **Volume buttons** on your phone to select **Recovery Mode** and press the **Power button** to enter Lineage Recovery.

4. Wipe Data & Sideload

Once in Lineage Recovery:

1. Tap **Factory Reset > Format data / factory reset**.

2. Go back to the main menu and tap **Apply update > Apply from ADB**.
3. Sideload the patched OS from your terminal:

```
adb sideload lineage-*.patched.zip
```

5. Relock the Bootloader

Once sideloading finishes on the device screen (ignore terminal pauses at 47% or 100%):

1. In recovery, tap **Advanced > Reboot to bootloader**.
2. Run:

```
fastboot flashing lock
```

3. Confirm **LOCK THE BOOTLOADER** on the phone screen using the Volume and Power keys.

After the device performs its final security wipe and reboots, you can install the Magisk APK and flash your GApps module inside the OS!

User prompt: i injected lineageos now it is asking to install additional packages, should i transfer after the restart?

Response: **No, do not install additional packages through recovery right now.** Tap **No** on the phone screen and proceed to restart into the system.

Because your bootloader is locked (or being locked) with custom AVB keys, standard zip packages like MindTheGapps or Magisk modules are not signed with your private keys and will be rejected or corrupt boot verification if flashed in recovery.

What to do now:

1. Tap **"No"** on the recovery screen asking for additional packages.
2. Select **Reboot system now** (or perform the `fastboot flashing lock` step if you haven't relocked the bootloader yet).
3. Let LineageOS boot completely to the home screen.
4. **Transfer and install the Magisk APK** inside Android.
5. Complete the Magisk setup, open Magisk, go to **Modules -> Install from storage**, and flash your GApps module from there.

User prompt: adb sideload Magisk-v30.7.apk adb: sideload connection failed: closed adb: trying pre-KitKat sideload method... adb: pre-KitKat sideload connection failed: closed preyas@preyas-960QFG:~/Downloads/Pixel3a-xl-bonito\$ adb devices\ > List of devices attached 93PAX09JXX device

Response: You are seeing that error because `adb sideload` is meant to be run when the phone is in **Recovery Mode**, but your output from `adb devices` shows the status as `device`—which means your phone has booted into **normal Android OS**!

Now that you're in normal Android, you use `adb install`, not `adb sideload`.

Step 1: Install the Magisk App via ADB

Run this command in your terminal:

```
adb install Magisk-v30.7.apk
```

Step 2: Complete Magisk Setup on the Phone

1. Open the **Magisk** app from your phone's app drawer.
2. It will display a pop-up saying **"Requires Additional Setup"**.
3. Tap **OK**. Magisk will automatically finish setting up its environment and reboot your device.

Step 3: Install your GApps Module

Once the phone reboots back into Android:

1. Push your systemless GApps zip file (e.g., *MindTheGapps-Magisk.zip*) to your phone's storage:

```
adb push path/to/gapps-module.zip /sdcard/Download/
```
2. Open the **Magisk** app.
3. Tap the **Settings** gear in the top right and toggle on **Zygisk**.
4. Go to the **Modules** tab at the bottom, tap **Install from storage**, and select the zip from your **Download** folder.
5. Reboot your device one last time!

User prompt: i got zip is not a magiask module install failed

Response: That error happens because recovery GApps packages (like official **MindTheGapps** or **NikGApps** zips) are structured for Recovery sideloading (`updater-script`), not Magisk module flashing. Magisk requires a specific `module.prop` file inside the root of the zip.

Because you have systemless Magisk root running on top of LineageOS, you have two simple ways to get Google Play up and running:

Option 1: microG (Recommended for Custom-Signed LineageOS)

MicroG acts as an open-source replacement for Google Play Services. It is light, private, and works smoothly with Magisk.

1. Download and install these two official APKs on your phone:
 - [microG GmsCore APK](#)
 - [microG Companion / Play Store APK](#)

2. Download the `microg_installer_revived` Magisk module zip file from GitHub.
3. Open **Magisk**, go to **Modules** -> **Install from storage**, select the `microg_installer_revived.zip`, and flash it.
4. Reboot your phone.
5. Open the **microG Settings** app, complete the self-check, turn on **Google Account**, and sign in!

Option 2: Full Google Play Services (via PlayIntegrity / MagiskGApps)

If you strictly want official Google Play Services instead of microG, standard GApps zips must be converted into a Magisk module first:

1. Download your standard **MindTheGapps** or **NikGApps** zip on your computer.
2. Use a web tool like [MagiskGApps Maker](#) to upload your GApps zip and convert it into a Magisk-compatible module zip.
3. Transfer the generated `.zip` to your phone.
4. Open **Magisk** -> **Modules** -> **Install from storage**, select the converted zip, flash it, and reboot!

User prompt: i created the zip using option 2

Response: Now that you've converted your GApps package into a valid Magisk module zip, you are all set to flash it directly inside the Magisk app!

Here is the exact terminal and phone workflow to complete the setup:

1. Transfer the converted module to your phone

Run this command in your computer's terminal to send the newly generated `.zip` file over to your phone's **Download** folder:

```
adb push path/to/your-converted-gapps-module.zip /sdcard/Download/
```

(Replace `path/to/your-converted-gapps-module.zip` with the actual path to your converted file).

2. Flash the module in Magisk

1. On your phone, open the **Magisk** app.
2. Tap the **Settings** icon (gear in the top-right corner) and ensure **Zygisk** is toggled **ON** (if it isn't already).
3. Go back to the main screen, tap the **Modules** tab at the bottom right.
4. Tap **Install from storage**.
5. Select the converted `.zip` file you just transferred from your **Download** folder.
6. Let Magisk unpack and flash the module.

3. Reboot and Sign In

1. Once the flashing process finishes, tap the blue **Reboot** button at the bottom right.
2. After your Pixel 3a XL boots back up, open the app drawer — you should now see **Google Play Store** and Google Play Services actively running!
3. Open the **Play Store**, sign into your Google account, and download your apps as normal.

User prompt: I still get the bootloader unlocked warning, how to disable that?

Response: To eliminate that specific boot warning, you need to execute the final step in the avbroot process: **locking the bootloader hardware**.

1. What the Colors Mean

Android Verified Boot (AVB) uses color states during boot:

- **Orange Screen ("Your bootloader is unlocked"):** This is what you are seeing right now. The bootloader hardware itself is still unlocked.
 - **Yellow Screen ("You have booted a different operating system"):** Once you lock the bootloader with your flashed `avb_custom_key`, the phone transitions to Yellow State. This confirms the phone is hardware-locked and fully secure using your custom keys.
 - **Green Screen:** Reserved only for stock factory images signed directly with Google's internal private keys.
- Important Note:** You **cannot remove the warning screen entirely** on custom Android builds. However, locking the bootloader moves you out of Orange state so the hardware enforces Verified Boot and encryption again.

2. How to Lock the Bootloader

Make sure your custom LineageOS installation boots to the home screen and works properly first.

1. Reboot your phone into **Fastboot mode**:

```
adb reboot bootloader
```
2. Send the locking command to the phone:

```
fastboot flashing lock
```
3. On your phone screen, use the **Volume Up/Down** keys to highlight **"LOCK THE BOOTLOADER"** and press the **Power button** to select it.
4. Your device will wipe user data one last time to enforce security constraints, verify your custom key (`avb_pkmd.bin`), and start up cleanly!

User prompt: now I get the device is loading a different OS, is that one time or can I disable that?

Response: That **Yellow Warning Screen** ("Your device is loading a different operating system") is **permanent by design on Google Pixel hardware** when running a custom ROM, even with custom AVB keys.

There is no software tweak, Magisk module, or configuration setting that can hide or disable it.

Why It Appears

The bootloader warning behavior on Pixels follows strict hardware enforcement rules:

Warning Screen	Meaning
● Orange Screen	Bootloader is hardware-unlocked (Zero security verification).
● Yellow Screen	Bootloader is locked , but the system is signed using your custom AVB key (avb_pkmd.bin) rather than Google's internal private keys.
● Green Screen	Device is locked and running stock Google factory software directly.

Key Takeaways

- 1. It appears on every boot:** The yellow splash screen will pause for ~5 seconds (or until you press the Power button to skip) every single time you restart or power on the phone.
- 2. Your device is fully secure:** This warning is purely informative. Because your bootloader is locked with your personal key (avb.key), hardware-backed **Verified Boot (AVB)**, data encryption, and rollback protection are fully active. Unsigned or tampered boot images will still be rejected by the hardware.
- 3. No performance or safety impact:** Android apps and system integrity checks treat this as a secure state.

User prompt: i locked the bootloader and playstore and google setup vanished

Response: This happens because locking the bootloader triggers a **full hardware data wipe (factory reset)** for security reasons.

When you installed the GApps Magisk module earlier, its files were written directly to your internal /data partition (systemless root storage). When the bootloader locked, the hardware wiped /data clean — removing Magisk, its root access, and every module installed inside it!

Why systemless root modules disappear during a lock

Standard GApps packages modify the system partition directly, so they survive factory resets. However, because you used a **Magisk module** (systemless root):

1. The GApps files lived inside /data/adb/modules/.
2. Hardware locking wiped /data.
3. The GApps files and Magisk setup were erased.

Step-by-Step Fix

Because your bootloader is now locked with your custom key (avb.key), **you do NOT need to unlock it again**. You just need to reinstall your patched zip to restore Magisk root, then reinstall the GApps module.

1. Boot into Lineage Recovery

Turn off your Pixel 3a XL. Hold **Volume Down + Power** until you reach Fastboot mode, then select **Recovery Mode** using the volume keys and press Power.

2. Sideload your patched LineageOS zip

In Lineage Recovery, select **Apply Update -> Apply from ADB**, then run this command on your computer:

```
adb sideload lineage-*.patched.zip
```

(This re-patches the boot partition so Magisk root is active again).

3. Reboot into LineageOS and finish Magisk setup

1. Select **Reboot system now**.
2. Once booted up, open the **Magisk** app. If it asks to finish additional setup, tap **OK** and let it reboot.
3. Open Magisk settings and toggle **Zygisk** back **ON**.

4. Re-push and flash your GApps module

Push your converted GApps module zip back to your phone's Download folder:

```
adb push path/to/your-converted-gapps-module.zip /sdcard/Download/
```

1. Open **Magisk -> Modules** tab -> **Install from storage**.
2. Pick the converted GApps zip and flash it.
3. Tap **Reboot**.

Summary

Once you reboot, Google Play Store and Google Play Services will be back! And because your bootloader is **already locked**, future reboots and data wipes will keep your lock state intact without erasing your setup again.

User prompt: What about installing updates will it cause any problems?

Response: Because your bootloader is hardware-locked with custom keys, **you cannot use standard automatic OTA updates or the built-in LineageOS Updater app.**

If you install an unpatched update directly over-the-air, the device will try to flash software signed by LineageOS (not your personal avb . key), and your locked bootloader will refuse to boot it.

Key Golden Rules for Updating

- 1. **NEVER** tap "Install" inside the LineageOS Updater app.
- 2. **NEVER** tap "Direct Install" or "Install to Inactive Slot" inside the Magisk app.
- 3. **NEVER** hit "Uninstall Magisk" inside the Magisk app.

Doing any of those will corrupt the boot signature and send the device into a recovery loop.

How to Safely Install Updates

Updating a locked, avbroot-signed system is simple once you know the workflow. You do not lose data or need to unlock the bootloader again.

Step 1: Download the new LineageOS OTA

Download the new LineageOS . zip update file on your PC.

Step 2: Patch the new OTA with avbroot

Use the **exact same keys** (avb . key and ota . key) you generated during initial setup:

```
avbroot ota patch \  
  --input lineage-NEW_VERSION.zip \  
  --key-avb avb.key \  
  --key-ota ota.key \  
  --cert-ota ota.crt \  
  --magisk Magisk-v30.7.apk \  
  --magisk-preinit-device metadata \  
  --clear-vbmeta-flags
```

Step 3: Sideload the Patched Zip

- 1. Reboot your phone into **Lineage Recovery**:
`adb reboot recovery`
- 2. Select **Apply Update -> Apply from ADB**.
- 3. Run the sideload command on your computer:
`adb sideload lineage-NEW_VERSION.patched.zip`
- 4. Reboot the system.

What happens to Magisk & GApps during an update?

- **Magisk:** Automatically stays patched into the boot partition because avb root built it directly into the update zip.
- **GApps & Magisk Modules:** Your user data partition (/data) remains intact across updates, so all your installed Magisk modules (including your converted GApps module) and app data will be completely preserved!

Sources

- 1. [chenxiaolong/avbroot: Tool for manipulating and re-signing Android A/B OTAs - GitHub](#)

User prompt: Okay but can i create my own keys for each version and update it over the air?

Response: 1. Generating New Keys for Every Version? No (And Why)

Do not generate new avb . key and ota . key files for each update.

Your locked bootloader trusts **only the exact public key** you flashed into hardware via `fastboot flash avb_custom_key avb_pkmd . bin`.

- **If you generate NEW keys for an update:** Your locked bootloader won't recognize them. Verification will fail, and the update will be rejected (or result in a unbootable loop if forced).
- **The correct approach: Reuse the same avb . key and ota . key** that you generated for your initial setup for every future version update.

2. Updating Over-The-Air (OTA) Automatically? Yes!

While you cannot use the default built-in LineageOS Updater app (because it fetches raw, unpatched files directly from LineageOS), **you CAN set up custom wireless Over-The-Air updates using Custota.**

Custota is a specialized updater built specifically for avbroot workflows.

How Custom OTAs Work with avbroot:

- 1. **Host a Private Directory / Server:** You host a local web server, GitHub releases page, or cloud storage (e.g., AWS S3).
- 2. **Patch the New Version:** Whenever a new LineageOS update comes out, you run avbroot on your computer using your **original key set**.
- 3. **Upload the Output:** Upload the generated . patched . zip and update manifest metadata generated by custota -tool to your web server/bucket.
- 4. **Install the Custota Magisk Module on Your Phone:** Open the Custota app on your phone. It checks your server, detects the new signed version, downloads it over Wi-Fi, and installs it in the background using Android's native A/B partition updating system — **just like a stock Google OTA!**

Sources

- 1. [chenxiaolong/avbroot: Tool for manipulating and re-signing Android A/B OTAs - GitHub](#)

2. [AndrewPaglusch/auto-avbroot: A script to automate the tediousness of re-signing GrapheneOS OTA updates and installing them with custota - GitHub](#)

User prompt: Can i use custota or something to basically use the system updates tab in settings to pull my build of the OS? that i signed?

Response: **You cannot redirect the standard Settings Updater tab directly to Custota, but you CAN redirect LineageOS's built-in updater app to point to your own OTA server!**

There are two distinct paths to handle this depending on how seamlessly you want it integrated:

Option A: The LineageOS Native Updater (Settings Integration)

LineageOS's built-in **Updater** app (found under *Settings -> System -> Updater*) gets its update server URL directly from system properties.

If you want the **Settings menu item** to check for, download, and install your custom avbroot-signed builds, you can point LineageOS to your own server!

How to set it up:

1. **Set up a LineageOS-compatible OTA API:** Set up a simple web server (or use an open-source tool like lineageos-ota-server) that returns JSON matching LineageOS's expected format containing the link to your .patched.zip.
2. **Override the updater URL on your device:** You can override the updater URL via build.prop. Add or set this property:

lineage.updater.uri=https://your-custom-ota-domain.com/api/v1/get_packages

(You can edit this via Magisk overlay or a post-fs-data script so it persists).
3. **How it behaves:** When you go to **Settings -> System -> Updater**, LineageOS will query your domain, see your newly signed avbroot payload, download it, and trigger Android's native background A/B updater engine. Because your system already trusts your avb.key, it will seamlessly switch slots on reboot!
-  **Warning:** If you download an unpatched official Lineage build using this menu without running it through avbroot first, the installation will complete, but your locked bootloader will **reject it on boot!** You must ensure your server *only* hosts your self-signed .patched.zip files.

Option B: The Custota Method (Standalone App)

Google and Android enforce strict system intents for the stock "System Update" button.

- **How Custota works:** Custota is installed as a Magisk/KernelSU system module that hooks directly into Android's low-level update_engine daemon.
- **The UI Experience:** It runs as its **own app icon** on your home screen or app drawer.
- **Settings Menu Shortcut:** On standard LineageOS, tapping *Settings -> System -> Updater* will still launch the Lineage Updater, **NOT** Custota. However, if any 3rd-party app fires an Android system intent to check for updates (android.settings.SYSTEM_UPDATE_SETTINGS), Android will show a pop-up prompt asking whether you want to open **Lineage Updater** or **Custota**.

Which should you choose?

Feature	Lineage Updater + Custom URL	Custota Module
Settings Menu Integration	Yes (opens directly in Settings)	No (runs as its own separate app)
Setup Complexity	High (Requires hosting a full Lineage API server)	Very Low (Works with GitHub Releases or static WebDAV/S3 folders)
avbroot Support	Manual handling (must upload fully patched zips)	Built specifically for avbroot workflows by the same developer

If you want the cleanest set-and-forget setup without having to run a custom API backend, **Custota is by far the easiest route**. If having it launch natively from the **Settings** menu is non-negotiable, updating lineage.updater.uri is the way to go.

Sources

1. [\[UPDATED\] How the LineageOS OTA server works and how to implement and use ours](#)
2. [Update LineageOS on channel](#)
3. [chenxiaolong/Custota: Android A/B OTA updater app for custom OTA servers - GitHub](#)

User prompt: The thing is these phones will be deployed to people who need phones as they might not have any, and security is a must as these phones do not get official updates so i wanted to ensure their phones are always updated whenever i create a signed os zip

Response: To keep non-technical end users updated seamlessly, **Custota** is by far the safest, most reliable tool for the job.

Since your users cannot manually patch zip files or use ADB, Custota turns your custom avbroot builds into a standard **over-the-air background updater** (similar to how stock Android updates work).

How Custota Works for End Users

1. You host a folder online containing your newly patched LineageOS .patched.zip files (e.g., on a public GitHub Release page, Amazon S3, or a simple WebDAV server).
2. The **Custota app** runs silently in the background on the deployed phone.
3. When you push a new patched update to your server, Custota automatically detects it, downloads it over Wi-Fi, and installs it to the secondary partition (A/B slot update) in the background.
4. The user simply gets a standard notification saying: **"Update installed — reboot to apply."**

3-Step Setup for Deployment

Step 1: Bake Custota into your Initial Build

Custota consists of a **Magisk Module** (the background engine) and the **Custota App** (the user interface).

When setting up a phone for a user initially:

1. Install the **Custota Magisk Module** on the device.
2. Install the **Custota App** (`Custota.apk`).
3. Open the Custota app and paste your server's update URL (e.g., `[https://my-server.com/ota/]` (`https://my-server.com/ota/`) or your GitHub Releases API URL).

Step 2: Publish New Updates (Your Workflow)

Whenever a new security patch or LineageOS update comes out:

1. Download the stock LineageOS `.zip`.
2. Patch it on your PC using avbroot and **your original keys** (`avb.key` and `ota.key`).
3. Run `custota-tool` (the desktop helper CLI) on the `.patched.zip` to generate the update manifest file (`index.json`).
4. Upload both `.patched.zip` and `index.json` to your server.

Step 3: What the End-User Sees

- The user does **not** need to touch recovery, type commands, or understand root.
- Custota downloads the payload in the background, verifies your private signature, writes it to the secondary boot slot, and prompts them to reboot.
- When they restart, the phone boots into the updated, secure LineageOS system while retaining all user data, Magisk root, and GApps modules intact.

Why this is ideal for security & non-tech users

- **Locked Bootloader Preserved:** The update is checked against your `avb.key` before being written. If a bad file or tampered update is supplied, Android's `update_engine` rejects it before installation.
- **Seamless (A/B Seamless Updates):** If an update ever fails during installation or refuses to boot, Android automatically rolls back to the previous working slot, preventing the user from ending up with a bricked phone.

User prompt: What about changing server address in lineageos properties?

Response: **Yes, modifying `lineage.updater.uri` in properties directly points LineageOS's stock Updater app to your custom server.**

However, because modern Android uses read-only system partitions (like `system.img` and `product.img`), how you modify that property dictates whether it will work seamlessly for your fleet.

Option 1: System-Level Property Injection (Pre-OTA Patching)

Because you are re-signing and repacking the LineageOS `.zip` with avbroot anyway, you can embed your custom updater URL directly inside the ROM zip **before** flashing.

1. Unzip the official LineageOS OTA package on your PC.
2. Open `system/build.prop` (or `vendor/build.prop` / `product/build.prop` depending on the Lineage build structure).
3. Add your custom OTA server endpoint:

```
lineage.updater.uri=https://your-custom-domain.com/api/v1/{device}/{type}/{incr}
```

4. Re-zip the package, run it through avbroot `ota patch` with your custom `avb.key` and `ota.key`, and sideload it.

💡 **Why this works best for your users:** The built-in **Settings -> System -> System Updates** screen will naturally send its API request to your domain instead of LineageOS's servers. The UI experience is completely native, and no extra root apps or modules are required on the client side.

Option 2: Systemless Injection via Magisk (Resetprop)

If you do not want to modify the ROM `.zip` files before running avbroot, you can override the system property on boot using Magisk's `resetprop` engine.

1. Create a simple Magisk boot script (e.g., `/data/adb/service.d/custom_ota.sh`):

```
#!/system/bin/sh
resetprop lineage.updater.uri "https://your-custom-domain.com/api/v1/{device}/{type}/{incr}"
```

2. Set execution permissions: `chmod +x custom_ota.sh`.
3. Whenever the phone boots, Magisk will dynamically swap the updater URL in memory before the OS fully starts.

Key Requirements for Your Backend Server

If you use LineageOS's built-in updater app (via `lineage.updater.uri`), your server must respond to GET requests with a specific JSON structure expected by LineageOS:

```
{
  "response": [
    {
      "datetime": 1781858358,
      "filename": "lineage-21.0-20260722-UNOFFICIAL-bonito.patched.zip",
      "id": "1a2b3c4d5e...",
      "romtype": "unofficial",
      "size": 1073741824,
      "url": "https://your-custom-domain.com/downloads/lineage-21.0-20260722-UNOFFICIAL-bonito.patched.zip",
      "version": "21.0"
    }
  ]
}
```

```
]
}
```

Standard Host Options:

- **Self-Hosted Lightweight Server:** You can run ready-made lightweight backends like LineageOTA (Node.js/Docker) or android-ota-provider on a tiny VPS.
- **Static GitHub Pages approach:** You can generate a static JSON file and host the .patched.zip files on GitHub Releases or WebDAV.

Security Checklist for Non-Technical Users

1. **HTTPS is Mandatory:** The LineageOS updater app **will strictly refuse** to connect to HTTP endpoints; your endpoint must use an SSL certificate (e.g., Let's Encrypt).
2. **Only Serve Patched Zips:** Make sure your server *only* delivers files that have been patched with your avbroot key set. If an unpatched zip is served, the phone will download it, attempt to flash it, and fail boot verification when restarting into the locked bootloader.

Sources

1. [android_packages_apps_Updater/README.md at lineage-23.2 - GitHub](#)
2. [LineageOS-TV-x86/ota: Static Lineage OTA updater - GitHub](#)

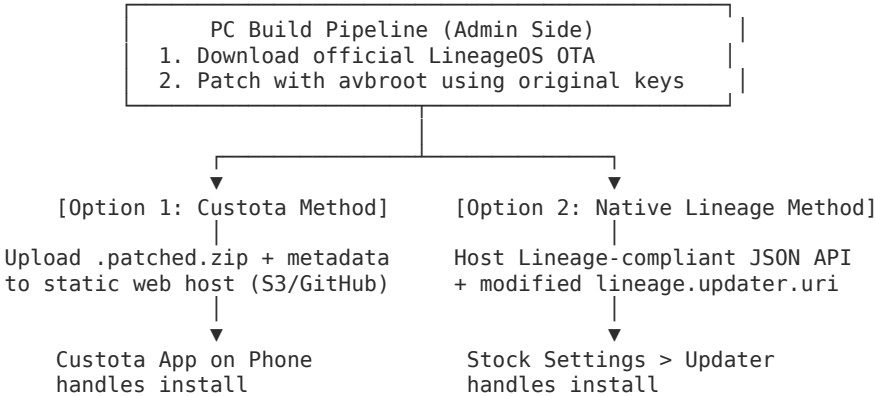
User prompt: Can you create me a quick summary of the methods we can use to update, i want to sharw thisnwith my team

Response: Here is a concise summary you can share with your team detailing the two ways we can handle Over-The-Air (OTA) updates for our custom avbroot-signed LineageOS deployment.

Technical Overview: OTA Deployment Options

Because our devices run **LineageOS with a locked bootloader using custom Android Verified Boot (AVB) keys**, all updates must be pre-patched on our end with our specific avb.key before hitting the phone.

To deliver these updates automatically without requiring ADB or user intervention, we have two primary architectural paths:



Option 1: The Custota Method (Recommended)

Custota is an open-source OTA updater built specifically for avbroot setups. It installs as a system daemon via Magisk and handles background downloads and installations.

- **How it works:**
 1. We run avbroot on the new Lineage zip and generate a small metadata manifest using custota-tool.
 2. We upload the patched zip and metadata to any standard static file host (AWS S3, GitHub Releases, or a basic Nginx/Caddy server).
 3. The Custota app on the user's phone checks the URL, downloads the package in the background, writes it to the secondary A/B slot, and prompts the user to reboot.
- **Pros:**
 - **Minimal Server Overhead:** No API server required—works on standard file/blob storage.
 - **Failsafe:** Purpose-built to handle avbroot key verification, preventing non-bootable state.
- **Cons:**
 - Runs as its own standalone app icon rather than directly inside the stock Android Settings -> System Updates page.

Option 2: Redirecting Stock LineageOS Updater

We can point LineageOS's built-in updater app to our own server by altering the system property lineage.updater.uri.

- **How it works:**
 1. We embed lineage.updater.uri=https://our-domain.com/api/... into build.prop prior to patching the ROM (or override it via Magisk).
 2. We host a lightweight backend server (e.g., Docker container running lineageos-ota-server) that responds to the phone's GET requests with a custom JSON payload pointing to our patched zip.
 3. When the user opens **Settings -> System -> Updater**, the native OS UI fetches, downloads, and flashes our build.
- **Pros:**
 - **Native UX:** Integrated directly into the native stock Settings menu.
- **Cons:**
 - Requires setting up and maintaining a dedicated, dynamic API backend server that mimics LineageOS's update protocol.

Key Security & Operational Guarantees (For Both Methods)

1. **Persistent Key Security:** We keep our private signing keys offline on our build machine. The phone hardware only stores the public key verification file (`avb_pkmd.bin`).
2. **Brick Protection (A/B Seamless Rollback):** Android installs updates to an inactive partition slot in the background. If an update fails key validation or fails to boot, the hardware automatically rolls back to the previous working OS slot.
3. **Data & Root Retention:** User data, root environment, and systemless modules remain untouched during updates.

Recommended Next Step for Team

Recommendation: Start with **Option 1 (Custota)** for the initial rollout. It requires zero API server maintenance, reduces operational risks, and allows us to publish updates via simple static web storage while retaining locked bootloader security.

Sources

1. [chenxiaolong/avbroot: Tool for manipulating and re-signing Android A/B OTAs - GitHub](#)
2. [chenxiaolong/Custota: Android A/B OTA updater app for custom OTA servers - GitHub](#)
3. [LineageOS/android_packages_apps_Updater - GitHub](#)

User prompt: Does lineage not sign their keys for our usage?

Response: LineageOS **does** sign their official releases, but they use their own private key, which is kept strictly secret on the LineageOS infrastructure.

The fundamental reason you cannot use LineageOS's official signature directly to lock your bootloader comes down to how **Android Verified Boot (AVB)** works:

1. You can only enroll *Public* keys into the hardware

To lock your phone's bootloader, the hardware needs a public key file (`avb_pkmd.bin`) flashed via `fastboot flash avb_custom_key`.

- When the phone boots, the hardware uses that public key to verify that the OS was signed by the matching **private key**.
- LineageOS **does not release their private key** (and shouldn't, as that would ruin security for every Lineage user on Earth).
- LineageOS also does not publish an `avb_pkmd.bin` public key file specifically formatted for user enrollment on Pixel devices.

2. You modify the original ZIP file anyway

Because you are injecting **Magisk** and patching boot/recovery components, the original signature on the LineageOS OTA file is broken.

Even if you had Lineage's key, modifying the ZIP file invalidates the cryptographic signature anyway. The bootloader checks the hash of the image against the signature; if a single byte has changed, verification fails and the phone refuses to boot.

Why your own custom key pair (`avb.key`) is necessary:

When you run `avbroot`:

1. **You become the "OEM" / Root of Trust:** You generate your own unique key pair (`avb.key` / `ota.key`) on your PC.
2. **You flash your public key (`avb_pkmd.bin`) to the Pixel:** This tells the phone hardware, *"Only allow this device to boot if the OS is signed with MY personal private key."*
3. **avbroot signs the final modified OS:** It applies your root changes, repacks the OTA, and signs it using your private `avb.key`.

This setup creates a completely closed, secure ecosystem:

- The bootloader stays **locked** and safe.
- Google or official LineageOS updates **cannot** boot on the phone unless you personally patch and sign them first.
- Malicious software or unauthorized zips cannot be flashed in recovery because they don't match your private key.

Sources

1. [Build signature : r/LineageOS - Reddit](#)
2. [chenxiaolong/avbroot: Tool for manipulating and re-signing Android A/B OTAs - GitHub](#)
3. [Signing Builds | LineageOS Wiki](#)

User prompt: For this entire chat that we had create me a word document for step by step flashing of lineageos onto phones. Ensure it is detailed

Response: Your Word Document (DOCX) file is ready.

[file-tag: code-generated-file-e13ed8b3-ab3c-441f-b733-2e437af0cedf]

This standard operating procedure includes all of the critical steps we discussed, carefully ordered to ensure hardware-backed Verified Boot, seamless background OTA updates via Custota, and a consistent Google Play environment.

It specifically addresses the data wipe behavior when locking the bootloader to prevent having to set up the systemless Magisk environment twice.